Implementação e Análise de Algoritmos de Ordenação

Guilherme Macke Ebert, Lorenzo Kissmann Rota

[1] Universidade Federal do Pampa (UNIPAMPA)
Bagé – RS – Brasil

Abstract. *This paper presents the implementation and experimental evaluation of sorting algorithms.*

Resumo. *Este trabalho apresenta a implementação e avaliação experimental de algoritmos de ordenação.*

1. Introdução

Algoritmos de ordenação são procedimentos fundamentais na computação, utilizados para organizar conjuntos de dados de acordo com determinada ordem. Embora diferentes algoritmos resolvam o mesmo problema, suas implementações apresentam características distintas em relação ao tempo de execução, ao consumo de memória e ao comportamento diante de diferentes distribuições de entrada.

O estudo dessas diferenças permite compreender, na prática, como a complexidade computacional e as decisões de implementação influenciam o desempenho de um algoritmo.

Este trabalho tem como objetivo implementar e avaliar experimentalmente dez algoritmos de ordenação: Bubble Sort, Insertion Sort, Selection Sort, Shell Sort, Heap Sort, Merge Sort, Quick Sort, Counting Sort, Radix Sort e Bucket Sort.

A avaliação considera diferentes tamanhos de entrada e condições de organização dos elementos, buscando comparar o desempenho observado com as complexidades teóricas dos algoritmos.

Além da medição dos tempos de execução, foram realizados testes de referência para verificar o funcionamento das implementações diante de diferentes características dos vetores de entrada.

2. Fundamentação Teórica

Os algoritmos estudados podem ser divididos em dois grupos: algoritmos baseados em comparação entre elementos e algoritmos que utilizam propriedades específicas dos valores de entrada.

2.1. Algoritmos baseados em comparação

Os algoritmos baseados em comparação determinam a posição dos elementos por meio de operações de comparação e movimentação.

2.1.1. Bubble Sort

O Bubble Sort percorre repetidamente o vetor, comparando elementos adjacentes e trocando suas posições quando estão fora de ordem. O processo é repetido até que não sejam necessárias novas trocas.

A implementação utilizada possui detecção de vetor ordenado, interrompendo a execução antecipadamente quando uma passagem completa não realiza nenhuma troca.

2.1.2. Insertion Sort

O Insertion Sort percorre o vetor e insere cada elemento em sua posição adequada dentro da parte já ordenada. Seu desempenho depende significativamente da organização inicial dos dados.

2.1.3. Selection Sort

O Selection Sort procura o menor elemento da parte ainda não ordenada do vetor e o posiciona na próxima posição disponível. Esse processo é repetido até que todos os elementos estejam ordenados.

2.1.4. Shell Sort

O Shell Sort é uma generalização do Insertion Sort que permite a comparação e movimentação de elementos distantes entre si.

A implementação utiliza a sequência de intervalos de Knuth, definida pela recorrência:

$$h_{k+1} = 3h_k + 1$$

Assim, são obtidos intervalos como 1, 4, 13, 40, 121 e assim sucessivamente.

Inicialmente, é selecionado o maior intervalo da sequência menor que o tamanho do vetor. Em seguida, o vetor é organizado considerando elementos separados por esse intervalo.

A cada etapa, o intervalo é dividido por três, aproximando progressivamente os elementos de suas posições definitivas. O processo termina quando o intervalo chega a 1, realizando uma última ordenação semelhante ao Insertion Sort.

2.1.5. Heap Sort

O Heap Sort utiliza uma estrutura de heap para organizar os elementos. Na implementação, é utilizada uma função para construir e manter um Max-Heap, no qual cada elemento pai possui valor maior ou igual ao de seus filhos.

A ordenação é realizada a partir da construção do heap e da retirada sucessiva do maior elemento, que é colocado no final da região ainda não ordenada.

2.1.6. Merge Sort

O Merge Sort utiliza a estratégia de divisão e conquista. O vetor é dividido recursivamente em partes menores até que cada parte contenha apenas um elemento.

Em seguida, as partes são combinadas de maneira ordenada, até que todo o vetor esteja reorganizado.

2.1.7. Quick Sort

O Quick Sort também utiliza a estratégia de divisão e conquista. A implementação possui uma função de particionamento que seleciona o último elemento do intervalo como pivô.

Durante o particionamento, os elementos menores ou iguais ao pivô são posicionados à sua esquerda, enquanto os maiores são posicionados à sua direita.

Após essa etapa, o algoritmo é chamado recursivamente para ordenar as duas regiões resultantes.

A escolha fixa do último elemento como pivô constitui uma limitação da implementação, pois pode produzir partições muito desequilibradas em determinadas entradas, especialmente vetores já ordenados ou invertidos. Nessas situações, a profundidade da recursão pode crescer significativamente, podendo ocasionar estouro da pilha de chamadas.

2.2. Algoritmos não baseados em comparação

Diferentemente dos algoritmos anteriores, os algoritmos apresentados nesta seção utilizam características dos valores de entrada para realizar a ordenação.

2.2.1. Counting Sort

O Counting Sort utiliza um vetor auxiliar para contabilizar a quantidade de ocorrências de cada valor presente na entrada.

A partir dessas contagens, é possível determinar as posições dos elementos na sequência ordenada.

Sua aplicação depende de um intervalo de valores adequado, pois o espaço auxiliar utilizado está relacionado à amplitude desse intervalo, sendo esta sua maior limitação.

2.2.2. Radix Sort

O Radix Sort realiza a ordenação considerando individualmente os dígitos dos elementos, em vez de comparar diretamente os valores completos.

A implementação utiliza um procedimento próprio baseado no Counting Sort para ordenar os elementos conforme cada posição decimal.

2.2.3. Bucket Sort

O Bucket Sort distribui os elementos em diferentes baldes, de acordo com seus valores.

Na implementação utilizada, são criadas n listas encadeadas, sendo cada elemento inserido no balde correspondente ao índice obtido pela expressão:

$$índice = \lfloor n \cdot x \rfloor$$

em que n representa o número de baldes e x é o valor do elemento, pertencente ao intervalo $[0, 1)$.

Após a distribuição, cada balde é ordenado individualmente por meio de Insertion Sort. Finalmente, os elementos são reunidos na ordem dos baldes, produzindo o vetor ordenado.

2.3. Complexidade computacional

A Tabela 1 apresenta as complexidades assintóticas teóricas dos algoritmos avaliados.

Tabela 1. Complexidade assintótica dos algoritmos avaliados.

Algoritmo	Melhor caso	Caso médio	Pior caso
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Shell Sort	Dependente dos intervalos	Dependente dos intervalos	Dependente dos intervalos
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$
Radix Sort	$O(d(n + k))$	$O(d(n + k))$	$O(d(n + k))$
Bucket Sort	$O(n + k)$	$O(n + k)$	$O(n^2)$

Na tabela, n representa a quantidade de elementos, k representa o intervalo de valores ou a quantidade de posições auxiliares, conforme o algoritmo, e d representa a quantidade de dígitos processados pelo Radix Sort.

No Bucket Sort, o comportamento linear depende de uma distribuição suficientemente uniforme dos elementos entre os baldes. No pior caso, muitos elementos podem ser concentrados em um mesmo balde.

As complexidades apresentadas representam o comportamento teórico dos algoritmos e não necessariamente o desempenho observado em uma implementação específica.

3. Metodologia

3.1. Ambiente experimental

Os experimentos foram realizados em um computador equipado com processador AMD Ryzen 7 5825U e 16 GB de memória RAM.

O sistema operacional utilizado foi o Ubuntu 24.04.5 LTS. A implementação e execução dos algoritmos foram realizadas utilizando a plataforma Java, com as seguintes versões:

- Java: OpenJDK 21.0.12.1 (2026-08-18);
- Compilador: javac 21.0.12.1.

3.2. Implementação dos algoritmos

Os dez algoritmos foram implementados em Java, com suas respectivas funções de ordenação.

As implementações foram organizadas de maneira que os algoritmos pudessem ser executados sobre vetores de entrada compatíveis com suas características.

Para os algoritmos baseados em comparação, foram utilizados vetores de elementos do tipo `Int`. O Counting Sort e o Radix Sort receberam vetores de inteiros primitivos, enquanto o Bucket Sort recebeu vetores de números reais do tipo `double`.

Foram consideradas as particularidades de cada implementação, incluindo os intervalos de valores aceitos pelos algoritmos que utilizam estruturas auxiliares.

3.3. Testes de referência

Além dos experimentos de desempenho, foi desenvolvido um procedimento de testes integrado ao próprio programa.

Esse procedimento gera três vetores distintos de dez elementos para cada uma das seguintes condições:

- elementos em ordem aleatória;
- elementos em ordem crescente;
- elementos em ordem decrescente;
- elementos com várias ocorrências repetidas.

Para cada vetor, o programa registra sua configuração original, executa o algoritmo de ordenação correspondente e registra o resultado obtido.

Os registros são armazenados em um arquivo de texto denominado `testes.txt`, permitindo verificar visualmente se as implementações produzem sequências ordenadas nas diferentes condições avaliadas.

Esses testes possuem finalidade distinta dos experimentos de desempenho e não são utilizados no cálculo das estatísticas de tempo.

3.4. Características dos vetores de entrada

Os vetores utilizados nos experimentos de desempenho foram gerados automaticamente pelo programa.

Para os algoritmos que utilizam tipos de dados equivalentes, foram utilizadas cópias de uma mesma entrada, evitando que diferenças nos valores dos vetores interferissem na comparação dos tempos de execução.

Para os algoritmos com requisitos específicos, foram utilizados geradores próprios, respeitando as restrições de cada implementação.

O Counting Sort recebeu valores inteiros dentro de um intervalo limitado até 10.000, o Radix Sort recebeu inteiros não negativos e o Bucket Sort recebeu números reais pertencentes ao intervalo $[0, 1)$.

3.5. Tamanhos e repetições

Os experimentos foram organizados em rodadas de execução, sendo realizadas dez repetições para cada configuração experimental.

Os tamanhos dos vetores utilizados foram: 1.000, 10.000, 100.000 e 1.000.000.

Foram realizados testes com vetores de 10.000.000 de elementos, mas o tempo de execução de algoritmos com implementação $O(n^2)$ inviabilizou a conclusão do experimento em tempo viável.

A quantidade de repetições foi mantida para permitir a obtenção de medidas estatísticas e a análise da variação dos tempos de execução.

Tabela 2. Configurações dos experimentos de desempenho.

Tamanho do vetor	Condições de entrada	Repetições
1.000	Aleatória	10
10.000	Aleatória	10
100.000	Aleatória	10
1.000.000	Aleatória, crescente e decrescente	10 por condição

3.6. Procedimento de medição

O tempo de execução foi medido por meio dos recursos disponibilizados pela plataforma Java.

A geração e a cópia dos vetores foram realizadas antes da medição, de modo que o intervalo registrado correspondesse à execução do algoritmo de ordenação.

Os tempos obtidos foram armazenados em arquivo CSV, preservando os registros individuais de cada execução, incluindo a identificação do algoritmo, tamanho do vetor, condição de entrada e rodada experimental.

Quando uma execução não pôde ser concluída, a ocorrência foi registrada separadamente, permitindo identificar limitações das implementações e evitar que resultados incompletos fossem tratados como tempos válidos de ordenação.

3.7. Tratamento dos dados

Os tempos coletados foram organizados por algoritmo, tamanho do vetor e condição experimental.

Para cada configuração, foram calculadas as seguintes estatísticas:

- média;
- mediana;
- menor tempo observado;
- maior tempo observado;
- desvio-padrão.

Os registros individuais foram preservados em arquivos complementares, enquanto as estatísticas consolidadas foram utilizadas na elaboração das tabelas e gráficos apresentados neste trabalho.

4. Resultados

Esta seção apresenta os resultados obtidos nos experimentos, considerando os tempos de execução e as estatísticas calculadas.

4.1. Resultados estatísticos

Tabela 3. Tempo médio de execução dos algoritmos de ordenação (segundos).

Algoritmo	1.000 A	10.000 A	100.000 A	1M A	1M C	1M D
Bubble Sort	$4{,}7216 \times 10^{-3}$	$2{,}5194 \times 10^{-1}$	$4{,}2147 \times 10^{1}$	$9{,}6358 \times 10^{3}$	$2{,}4647 \times 10^{-3}$	$3{,}7259 \times 10^{3}$
Heap Sort	$1{,}3912 \times 10^{-4}$	$1{,}3628 \times 10^{-3}$	$2{,}1718 \times 10^{-2}$	$4{,}0339 \times 10^{-1}$	$1{,}3912 \times 10^{-1}$	$1{,}5836 \times 10^{-1}$
Insertion Sort	$1{,}2231 \times 10^{-3}$	$5{,}9586 \times 10^{-2}$	$9{,}6040 \times 10^{0}$	$2{,}1281 \times 10^{3}$	$6{,}3520 \times 10^{-3}$	$1{,}5729 \times 10^{3}$
Merge Sort	$2{,}3079 \times 10^{-4}$	$1{,}6440 \times 10^{-3}$	$3{,}3492 \times 10^{-2}$	$2{,}2714 \times 10^{-1}$	$1{,}2388 \times 10^{-1}$	$1{,}2258 \times 10^{-1}$
Quick Sort	$1{,}5250 \times 10^{-4}$	$1{,}0066 \times 10^{-3}$	$2{,}3360 \times 10^{-2}$	$2{,}0294 \times 10^{-1}$	SO	SO
Selection Sort	$4{,}3626 \times 10^{-3}$	$3{,}0018 \times 10^{-1}$	$3{,}9665 \times 10^{1}$	$5{,}9325 \times 10^{3}$	$3{,}3833 \times 10^{3}$	$3{,}2907 \times 10^{3}$
Shell Sort	$1{,}7453 \times 10^{-4}$	$1{,}6033 \times 10^{-3}$	$2{,}8687 \times 10^{-2}$	$6{,}5825 \times 10^{-1}$	$4{,}6964 \times 10^{-2}$	$7{,}4266 \times 10^{-2}$
Counting Sort	$3{,}1371 \times 10^{-5}$	$2{,}2797 \times 10^{-4}$	$8{,}3555 \times 10^{-4}$	$6{,}6246 \times 10^{-3}$	$4{,}1376 \times 10^{-3}$	$4{,}4505 \times 10^{-3}$
Radix Sort	$2{,}4561 \times 10^{-4}$	$1{,}0201 \times 10^{-3}$	$5{,}6415 \times 10^{-3}$	$5{,}5497 \times 10^{-2}$	$3{,}3468 \times 10^{-2}$	$2{,}6455 \times 10^{-2}$
Bucket Sort	$2{,}6020 \times 10^{-4}$	$1{,}5999 \times 10^{-3}$	$1{,}3952 \times 10^{-2}$	$1{,}8791 \times 10^{-1}$	$4{,}8153 \times 10^{-2}$	$2{,}9465 \times 10^{-2}$

A: aleatória; C: crescente; D: decrescente. SO: execução interrompida por estouro da pilha de chamadas (*Stack Overflow*). Cada média considera dez execuções concluídas com sucesso.

As tabelas apresentam as médias dos tempos de execução para cada configuração experimental. Os demais indicadores estatísticos são utilizados para complementar a análise da dispersão dos resultados.

4.2. Comparação gráfica dos tempos de execução

Os gráficos foram organizados em grupos de algoritmos com características de complexidade semelhantes, permitindo uma comparação mais clara dos resultados.

4.2.1. Algoritmos de complexidade quadrática

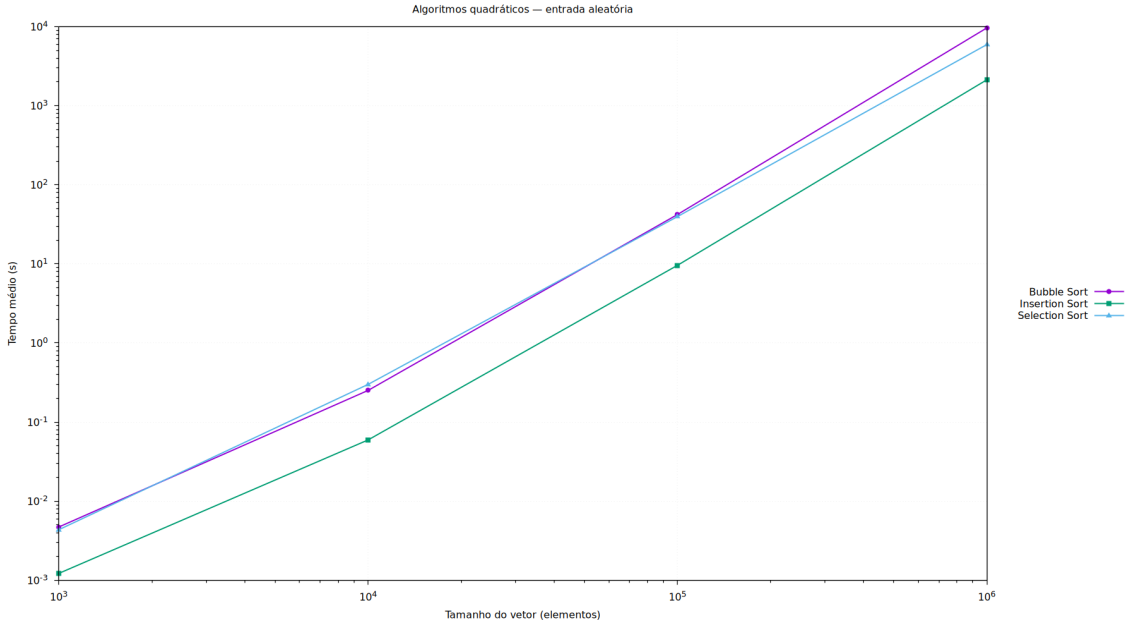


Figura 1. Algoritmos quadráticos com vetores de entrada aleatórios

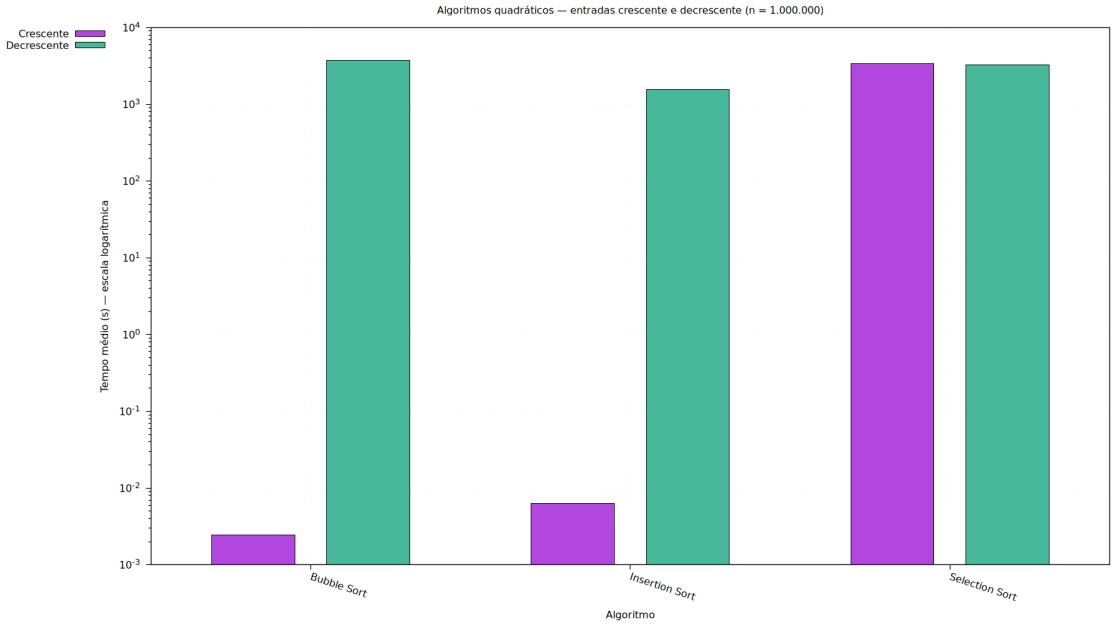


Figura 2. Algoritmos quadráticos com vetores de entrada ordenados

4.2.2. Algoritmos de complexidade $O(n \log n)$ ou dependentes da sequência de intervalos

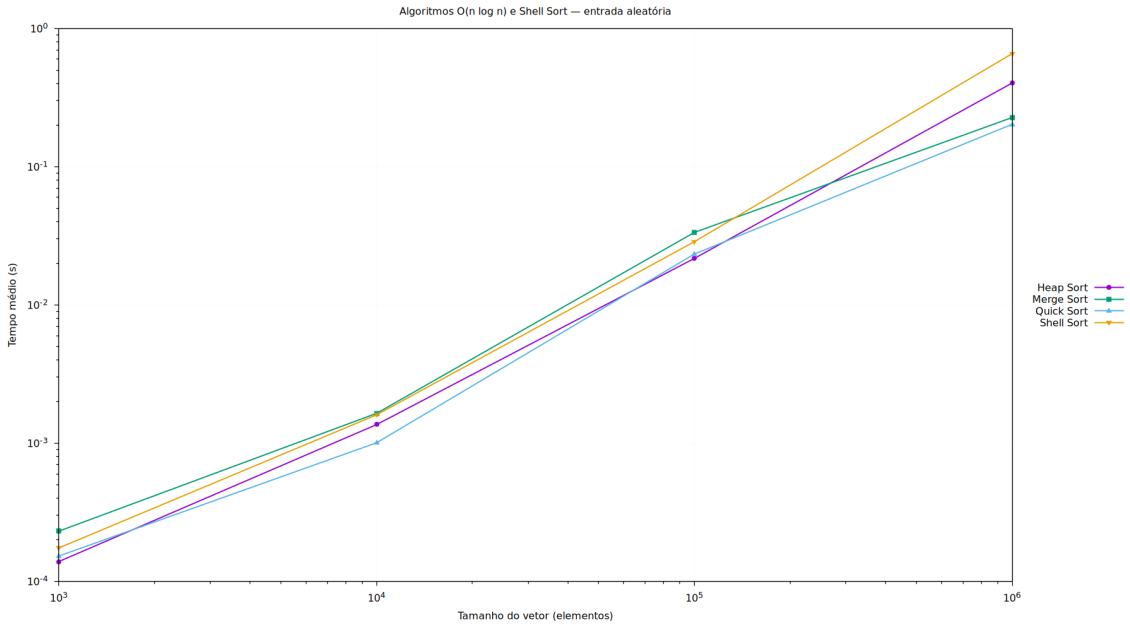


Figura 3. algoritmos logarítmicos com vetores de entrada ordenados

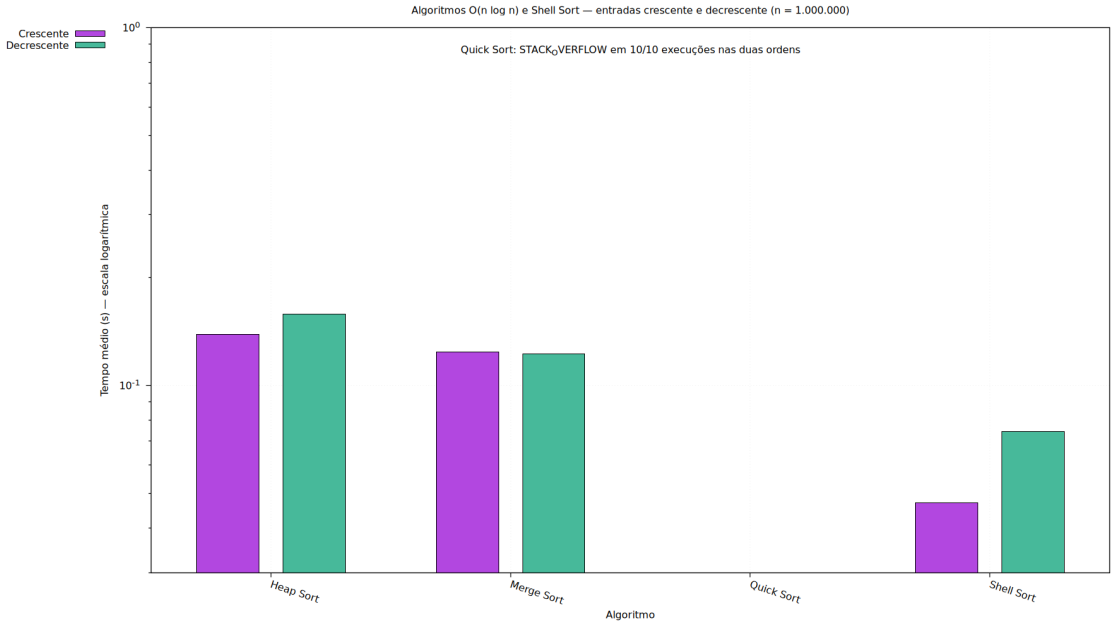


Figura 4. algoritmos logarítmicos com vetores de entrada ordenados

4.2.3. Algoritmos não baseados em comparação

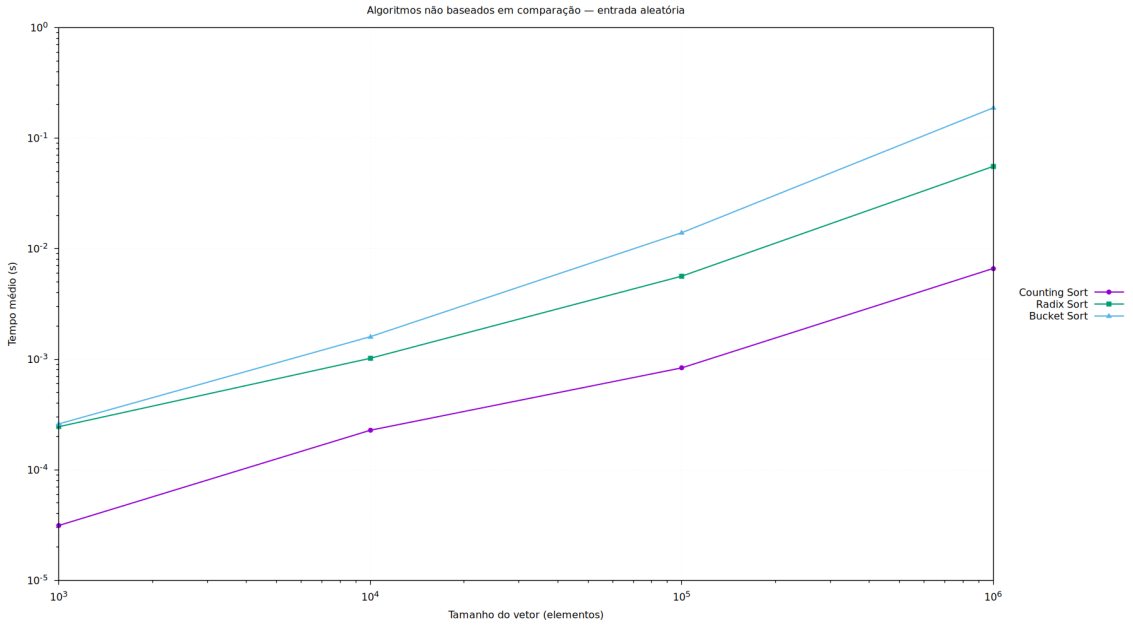


Figura 5. algoritmos não baseados em comparação com vetores de entrada aleatórios

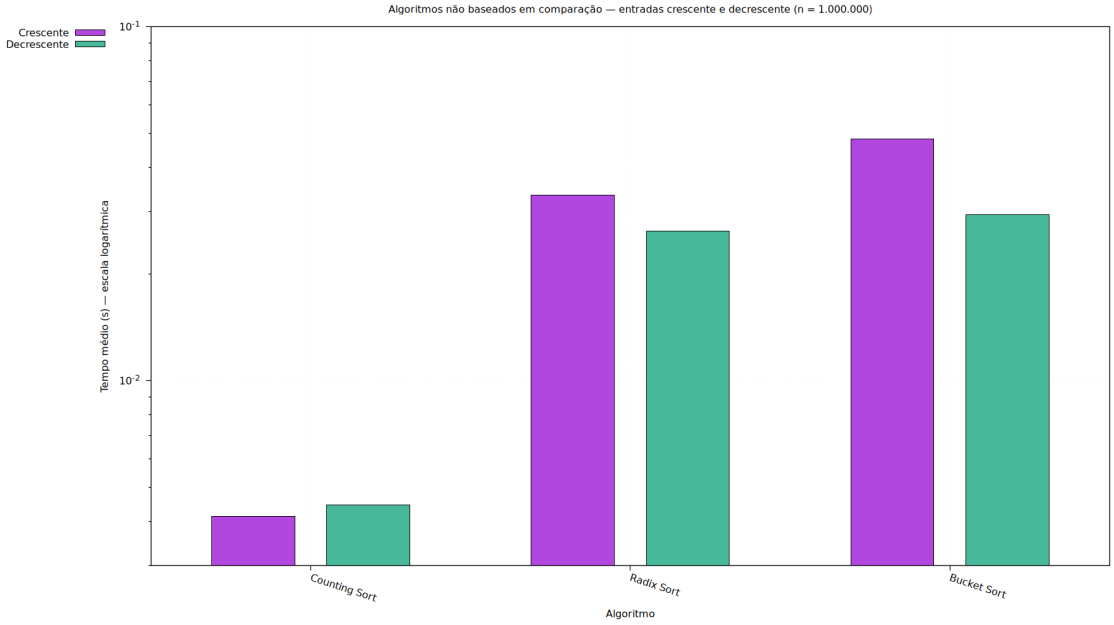


Figura 6. algoritmos não baseados em comparação com vetores de entrada ordenados

4.3. Variabilidade dos tempos de execução

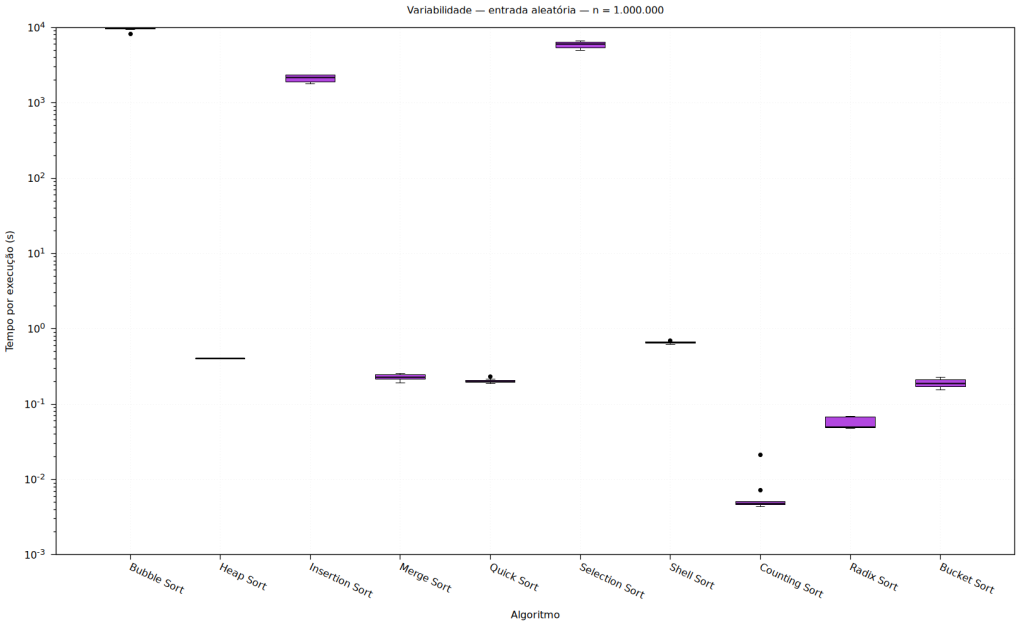


Figura 7. Variabilidade do tempo de execução entre 10 execuções de cada algoritmo com vetor aleatório

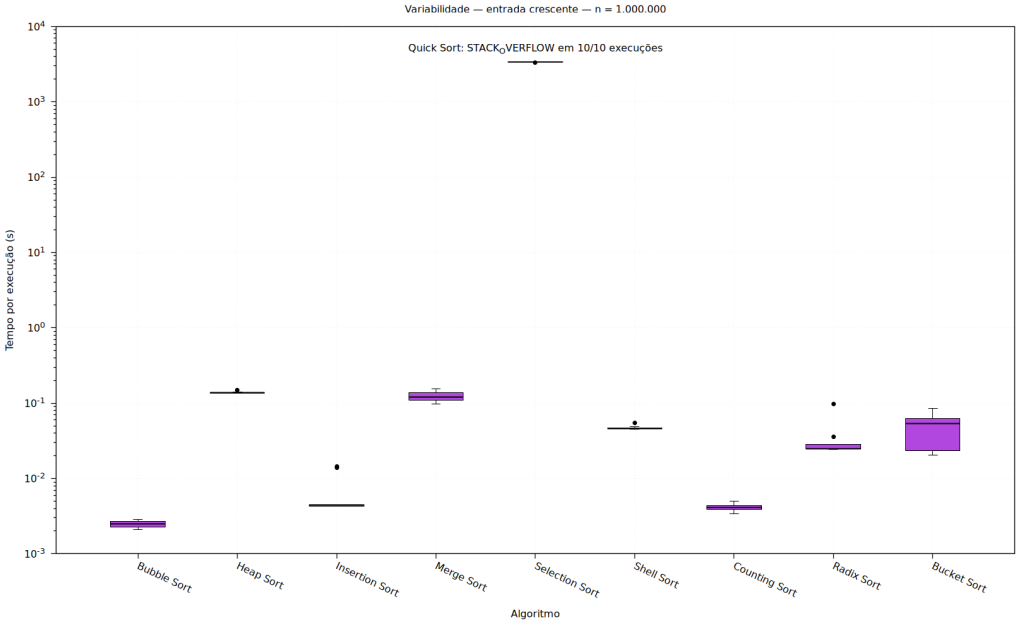


Figura 8. Variabilidade do tempo de execução entre 10 execuções de cada algoritmo com vetor crescente

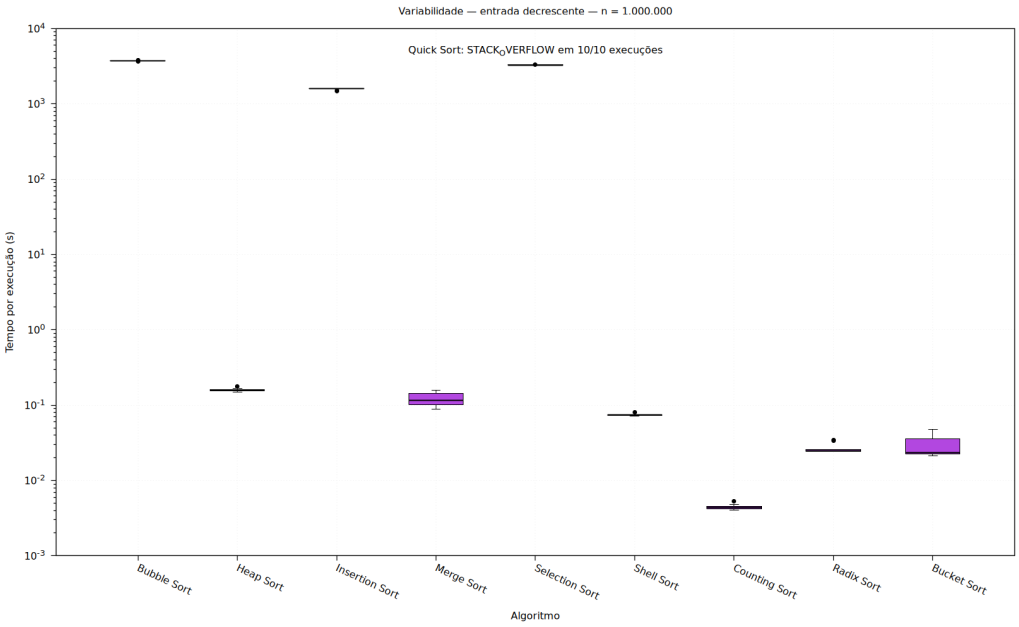


Figura 9. Variabilidade do tempo de execução entre 10 execuções de cada algoritmo com vetor decrescente

Os diagramas de caixa permitem observar a distribuição dos tempos de execução, destacando a mediana, a dispersão e a presença de possíveis valores atípicos.

Essa análise complementa a comparação baseada apenas nas médias, permitindo avaliar a estabilidade dos resultados obtidos.

5. Análise e Discussão

5.1. Influência do tamanho da entrada

O aumento do tamanho da entrada evidenciou diferenças expressivas entre os algoritmos. Os métodos quadráticos apresentaram crescimento muito acentuado, especialmente em entradas aleatórias. Para um milhão de elementos, Bubble Sort levou aproximadamente 9.636 segundos, Insertion Sort 2.128 segundos e Selection Sort 5.933 segundos.

Em contrapartida, Heap Sort, Merge Sort e Quick Sort concluíram a mesma ordenação em menos de meio segundo. Counting Sort apresentou o menor tempo entre todos os algoritmos, com aproximadamente 0,007 segundo.

Esses resultados são compatíveis com as complexidades teóricas: algoritmos $O(n^2)$ tornam-se rapidamente inviáveis para entradas grandes, enquanto métodos de complexidade inferior apresentam crescimento consideravelmente mais moderado.

5.2. Influência da organização dos dados

A organização inicial dos elementos teve influência significativa sobre alguns algoritmos. Bubble Sort e Insertion Sort foram especialmente favorecidos pela entrada crescente, apresentando tempos inferiores a 0,01 segundo para um milhão de elementos. Isso ocorre porque ambos possuem melhor caso linear quando a sequência já está ordenada.

Na entrada decrescente, entretanto, seus tempos aumentaram consideravelmente, chegando a aproximadamente 3.726 segundos no Bubble Sort e 1.573 segundos no Insertion Sort. O Selection Sort manteve tempos elevados nas três condições, como esperado devido à sua quantidade de comparações.

Heap Sort e Merge Sort apresentaram comportamento mais estável, com tempos próximos nas diferentes organizações. O Shell Sort também foi favorecido pelas entradas ordenadas. Já o Quick Sort não conseguiu concluir os testes com entradas crescentes e decrescentes, evidenciando sua sensibilidade à escolha do pivô.

5.3. Comparação entre algoritmos de complexidade semelhante

Entre Heap Sort, Merge Sort, Quick Sort e Shell Sort, o Quick Sort apresentou o melhor resultado na entrada aleatória, seguido de perto pelo Merge Sort. O Heap Sort teve desempenho um pouco inferior, enquanto o Shell Sort apresentou maior tempo nessa condição.

Apesar de possuírem complexidades semelhantes, diferenças nas operações realizadas, movimentações de elementos, acessos à memória e estratégias de implementação influenciam o desempenho prático. O Quick Sort, por exemplo, apresentou excelente resultado médio, mas não conseguiu lidar com determinadas organizações de entrada.

Entre os algoritmos quadráticos, o Insertion Sort foi o mais rápido na entrada aleatória, seguido pelo Selection Sort e pelo Bubble Sort. Na entrada crescente, porém, Bubble Sort e Insertion Sort apresentaram desempenho muito superior ao Selection Sort, devido à existência de casos favoráveis para os dois primeiros.

5.4. Comportamento dos algoritmos não baseados em comparação

Counting Sort, Radix Sort e Bucket Sort apresentaram tempos bastante inferiores aos algoritmos baseados em comparação. O Counting Sort foi o mais rápido do benchmark, levando aproximadamente 0,007 segundo para ordenar um milhão de elementos aleatórios.

Essa vantagem está relacionada às características dos dados: Counting Sort depende da amplitude dos valores, Radix Sort da quantidade de dígitos processados e Bucket Sort da distribuição dos elementos entre os baldes. Portanto, seu desempenho está diretamente ligado ao atendimento desses requisitos.

No caso do Bucket Sort, a implementação utiliza Insertion Sort para ordenar individualmente os elementos de cada balde. Assim, distribuições desequilibradas podem prejudicar seu desempenho. Os resultados obtidos demonstram a eficiência desses métodos nas condições testadas, mas não permitem generalizar sua superioridade para qualquer conjunto de dados.

5.5. Limitações das implementações

A principal limitação observada ocorreu no Quick Sort. A implementação utiliza o último elemento como pivô, estratégia que pode gerar partições extremamente desequilibradas quando a entrada já está ordenada ou inversamente ordenada.

Isso aumenta a profundidade das chamadas recursivas e pode levar ao estouro da pilha. De fato, todas as dez execuções com um milhão de elementos em ordem crescente e todas as dez em ordem decrescente falharam com `STACK_OVERFLOW`.

O problema poderia ser reduzido com estratégias alternativas de escolha do pivô, como seleção aleatória ou mediana de três elementos. Essa ocorrência demonstra que uma implementação pode apresentar excelente desempenho médio e, ainda assim, possuir limitações importantes em determinadas condições.

5.6. Comparação entre resultados experimentais e teóricos

De maneira geral, os resultados foram compatíveis com as expectativas teóricas. Os algoritmos quadráticos apresentaram crescimento acentuado, enquanto os métodos $O(n \log n)$ mantiveram tempos significativamente menores. Counting Sort, Radix Sort e Bucket Sort também demonstraram as vantagens esperadas quando suas condições de aplicação são favoráveis.

Entretanto, alguns resultados merecem atenção. O Bubble Sort e o Insertion Sort foram mais lentos na entrada aleatória do que na decrescente, embora esta última costume representar um caso desfavorável para ambos. Essas diferenças podem estar relacionadas a fatores da implementação ou do ambiente experimental e merecem ser consideradas na interpretação dos resultados.

Assim, os experimentos demonstram que a complexidade assintótica é fundamental para compreender o comportamento dos algoritmos, mas não explica isoladamente todas as diferenças observadas. A implementação e as características dos dados também exercem influência significativa sobre o desempenho.

6. Conclusão

Este trabalho apresentou a implementação e avaliação experimental de dez algoritmos de ordenação, considerando diferentes tamanhos e características dos vetores de entrada.

Os experimentos evidenciaram diferenças expressivas de desempenho. De modo geral, os algoritmos de complexidade linearítmica, como Merge Sort, Heap Sort e Quick Sort, apresentaram tempos significativamente menores que os algoritmos quadráticos em entradas grandes. Entre os algoritmos não baseados em comparação, o Counting Sort obteve os melhores resultados, embora seu desempenho dependa diretamente da faixa de valores utilizada.

Também foi possível observar a influência da organização dos dados. Bubble Sort e Insertion Sort apresentaram bons resultados em vetores já ordenados, enquanto o Selection Sort manteve tempos elevados mesmo nessas condições. O Quick Sort, por sua vez, revelou uma limitação importante de sua implementação: a escolha do último elemento como pivô levou a falhas por excesso de recursão nas entradas crescentes e decrescentes de maior tamanho.

Em geral, os resultados acompanharam as expectativas teóricas, mas também evidenciaram que a complexidade assintótica, isoladamente, não determina o desempenho observado. A estratégia de implementação, a distribuição dos dados e as características específicas de cada entrada também exercem influência significativa.

Assim, o trabalho demonstrou que a escolha de um algoritmo de ordenação deve considerar não apenas sua complexidade teórica, mas também o contexto de aplicação e as características dos dados que serão processados.

7. Disponibilidade dos resultados

Os registros individuais dos experimentos foram preservados em arquivos complementares, contendo os tempos de execução e as informações necessárias para identificar cada configuração experimental.

O arquivo de testes de referência também foi preservado, contendo os vetores originais e os resultados produzidos pelos algoritmos.

Os códigos-fonte, os dados experimentais e os scripts utilizados para gerar os gráficos estão disponíveis no repositório do projeto [1].

Referências

[1] Guilherme Ebert and Lorenzo Rota. Implementação e análise de algoritmos de ordenação, 2026. Disponível em: `https://github.com/lorenzokr/Trabalho_CD`. Acesso em: 3 nov. 2026.